Integrating DB & Application: Complex API (ORM)

Goal

Think of application + persistent objects

- Specify which objects persistent
- Interaction with DB largely(?) hidden









Main ideas

Abstraction: table = class, table row = object

Declarative specification generates

- SQL CRUD code
- Application code mapping application/DB data

Library manages data movement

Declarative specification

Hibernate

```
<hibernate-mapping>
    <class name="Student" table="Student">
        <id name="id" type="int" column="id">
            <generator class="native"/>
        </id>
        <property name="firstName" column="first_name"
                  type="string" />
        <property name="lastName" column="last_name"
                  type="string" />
    </class>
</hibernate-mapping>
```

Declarative specification

SQLAlchemy

```
class Student(Base):
    __tablename__ = 'Student'
    id = Column(Integer, primary_key=True, autoincrement=True)
    first_name = Column(String)
    last_name = Column(String)
```

Query

SQLAlchemy

```
query = db.select([census]).
          .where(db.and_(census.columns.state == 'California',
                         census.columns.sex != 'M'))
resultset = connection.execute(query)
```

SQLAlchemy

```
resultset = session.query(User, Document, DocumentPermission)
                   .join(Document)
                   .join(DocumentPermission)
                   .filter(User.email == 'john_smith@foo.com')
```

Data movement issues

Granularity

- Application-level traditionally accesses an attribute at a time.
- DB-level accesses one or more records at a time.

Consistency

- Are application & DB data guaranteed to be consistent?

Eager vs. lazy loading

Pros

- ☺ Often simplifies application programming
  - Generates lots of “boilerplate” SQL code.
  - Avoids SQL syntax
  - Fits in style of language
  - Allows static type checking of SQL
  - Limits or prevents SQL injection attacks
  - Ports to other RDBMS

Cons

- ☹ Doesn't always simplify learning curve
  - Often largely SQL in disguise
  - Still need to understand SQL concepts
  - Complex library
  - Every ORM is different.
- ☹ Adds overhead
- ☹ May encourage bad usage, e.g.,
  - SELECTing all fields
  - n+1 SELECTs problem

n+1 SELECTs Problem

ORM pseudo-code:

```
foreach emp in select_all_employees()
    print select_salary_of_employee(emp)
```

Avoid looping over SELECT!

Corresponds to:

```
SELECT * FROM Employee;

SELECT salary FROM Salary WHERE emp_id = 1
SELECT salary FROM Salary WHERE emp_id = 2
...
```

Instead of:

```
SELECT salary
FROM Salary AS s
INNER JOIN Employee AS e ON e.emp_id = s.emp_id
```

Cons

"ORMs make the easy even easier, and the difficult impossible. Writing good, high-performing SQL is hard enough as it is. Add an ORM and then you have to figure out how to say what you want through a mapping layer."